

Rückblick und Einstieg

Vom Treiber zum Platform-Device

4.01

Das Wissen aus Tag 3 bleibt unser Fundament

01	Kontext zuerst	Ausführungskontext, zulässige Locks und mögliche Wartezeiten bestimmen die API-Wahl.
02	Ownership klären	Ressourcen werden eindeutig übernommen und symmetrisch wieder freigegeben.
03	Grenzen prüfen	Eingaben, Längen, Offsets und Hardwareressourcen werden vor der Nutzung validiert.
04	Schnittstellen stabil halten	Semantik und Fehlercodes bleiben für Kernel und User Space nachvollziehbar.
05	Vorgänge abschließen	Jeder Request endet genau einmal – erfolgreich oder mit eindeutigem Fehler.

Dieses Fundament wenden wir heute auf Gerätebindung, Ressourcenverwaltung und Treiberlebenszyklen an.

Tag 4 verbindet I/O-Pfade mit dem Linux-Gerätemodell



Roter Faden

Ein Treiber verbindet Gerätebeschreibung, Ressourcen und Lebenszyklus zu einem überprüfbaren Vertrag.

Drei Leitfragen führen durch Tag 4

TERMINAL

```
$ DEV=sda # Beispiel anpassen
$ ls -l /sys/class/block/$DEV
$ readlink -f /sys/class/block/$DEV/device/driver
$ udevadm info --name=/dev/$DEV
```

- 01 Wie funktioniert das Matching?
Bus-Matching, compatible und probe()
- 02 Woher kommen die Ressourcen?
MMIO, Interrupts und Properties
- 03 Wie gelingt der sichere Rückbau?
Bind, Unbind, devm_*() und remove()

Ziel des Einstiegs

Vom gemessenen I/O-Pfad zum gebundenen Gerätetreiber